Greedy and Math

Topic 9 of 10

13 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Find the Largest Monotone Increasing Number

MEDIUM | asked 2026-07-04 |

Overview: Find the largest number no greater than n whose decimal digits are monotone increasing. Develop a linear-time greedy correction that revisits earlier decreasing boundaries and fills the remaining suffix optimally without introducing leading zeros.

```
def largest_monotone(n):
```

2. Choose optimal guesses for green-only Wordle

MEDIUM | asked 2025-10-14 |

Overview: This question evaluates probabilistic reasoning, expected-value optimization, information partitioning, and algorithmic efficiency in selecting guesses under partial (green-only) feedback; it belongs to the Coding & Algorithms domain, specifically search and probabilistic modeling, and emphasizes practical algorithm design and implementation rather than purely theoretical concepts.

```
def best_guess(words, weight=None):
```

3. Write SQL code to publish the Fibonacci series.

MEDIUM | asked n/a |

Write SQL code to publish the Fibonacci series.

```
def fibonacci(count):
```

4. Trip Expense Settlement

MEDIUM | asked n/a |

A group of friends go on a trip. Each friend may pay for others. Given payment records, compute minimal payments required to settle all debts.

```
def settle_greedy(records):
```

5. Check Digits and Combine Ranges

HARD | asked 2026-03-29 |

You are asked to solve two independent coding tasks. ### Task 1: Check Whether an Integer Reads the Same Backward Given an integer x, return true if its decimal representation reads the same from left to right and from right to left; otherwise return false. Requirements and edge cases: - Negative integers should return false because of the leading minus sign. - You may assume x fits in a signed 32-bit integer. - Try to solve it without converting the integer to a string.

```
def is_palindrome(x, base=10):
```

6. Find secret word with match-count feedback

HARD | asked 2026-02-11 |

Problem You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API: - matchCount(guess) -> int: when you submit a word guess from the list, the API returns the number of positions i such that guess[i] == secret[i] (i.e., exact character match at the same index). You are allowed to call matchCount **at most 10 times**.

```
def best_guess(candidates):
```

7. Wordle-Style 5-Letter Minimum-Guess Strategy

HARD | asked 2025-12-15 | algorithm-design, greedy

Phone screen coding: given a black-box oracle that tells you whether a specific letter is at a specific position in a hidden 5-letter word, design an algorithm to identify the word in as few oracle calls as possible. Hidden word: a 5-letter string from a..z. Oracle: query(pos: int, letter: char) -> bool returns whether position pos (0-indexed) holds letter. Return the hidden word. Primary metric: minimize the number of query calls.

```
def by_frequency(query):
```

8. Shuffle array using random integer API

HARD | asked 2025-12-08 |

You are given an array nums of length n representing a deck of distinct cards. You **do not have access to a built-in shuffle function, but you are given the following API: python rand_int(l, r) # returns a uniformly random integer in [l, r], inclusive ### Task Implement a function shuffle(nums) that randomly permutes the array **in place** such that **every one of the n! permutations is equally likely**.**

```
def shuffle(nums, rand_int):
```

9. Find maximum value in huge text file

HARD | asked 2025-12-08 |

You are given the path to a **very large** text file on disk. The file has the following properties: - Each line contains exactly one signed 64-bit integer (in decimal). - The file size can be tens or hundreds of gigabytes, so it **cannot** be fully loaded into memory. - You have standard file I/O APIs that allow you to read the file sequentially (e.g., line by line or in chunks). ### Tasks 1. - Assume you have limited RAM (much smaller than the file size).

```
def max_in_file(path):
```

10. Find All Bad Pairs Using runTest Oracle

HARD | asked 2025-11-27 | algorithm-design, math-reasoning

PhD intern coding round 2: given a black-box oracle runTest(S) that returns true iff a subset S contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible. Items t_1, ..., t_N are given. Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false. Output: every bad pair (t_i, t_j). Minimize total oracle calls. Examples: N=4, bad pairs = {(1,2)}.

```
def find_bad_pairs(items, run_test):
```

11. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

HARD | asked n/a |

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6.

```
def has_good_subarray(nums, k):
```

12. Implement a function to justify a list of words within a given width.

HARD | asked n/a |

Implement a function to justify a list of words within a given width.

```
def justify(words, width, last='left', measure=
  len, lines=None):
```

13. Min boards to cover roof holes

MEDIUM | asked 2026-02-12 |

Overview: This question evaluates competency in combinatorial optimization and problem modeling for coverage constraints, testing algorithmic reasoning, complexity analysis, and efficient representation of grid-based data.

```
def min_boards(holes, length):
```